ECE 385 Final Project Report

Matthew Wu (mzwu3), Sanjit Sriram (sanjits2)

Introduction:

For our final project, we created a digital synthesizer that can take USB keyboard inputs to play musical notes through square waves. This is done with a 16 voice PWM audio generator using the Urbana board's 3.5mm jack output. We also incorporated on-board loop recording functionality, as well as the ability to play pre-programmed songs that could be taken by converting MIDI files into our own format with a Python script we created. Additionally, we have a GUI, to show the user what notes are being played on an on screen keyboard, similar to Synthesia, with the layout of a standard piano keyboard. This also showed the notes being played by the loop recorder and the pre-programmed songs.

Explanation of Functions:

<u>Live Keyboard Play With Octave Shift:</u>

The two rows on the keyboard starting at tab and the tilde key serve as the white and black keys on a traditional keyboard/piano. At any given time, the keyboard supports a roughly two octave range, from Cx to B(x+1), where x is an octave number from 1-6. The F1-F6 keys are used to shift the octave up/down. The default is that the tab key is set to C4 (middle C), which is F4 on the keyboard. Clicking the F5 key, for example, would set the tab to C5. The full range of our keyboard with octave shifting is C1 to B7 on the standard piano. Up to four notes can be played at any given time, as that is the max capacity of the GPIO buffer that the MicroBlaze block writes to when outputting keycodes. Getting the keycodes is achieved using the

MicroBlaze block from lab 6.2 without modification. In summary, the MAX4231 USB host chip handles the interface with the port itself, and then sends data via SPI to the MicroBlaze, which in turn outputs it to a 32 bit AXI GPIO buffer. In the SV portion of the project, anything that needs access to keycodes does it by scanning the GPIO buffer.

Looper:

The looping function allows the user to loop a snippet of music that they play. Pressing the HOME key will start the recording. The user can then play anything on the keyboard. Pressing END will stop the recording and start looping what was played indefinitely. This can be used to create an accompaniment or a riff which the user can then play melodies over.

Pre Recorded Song Playback:

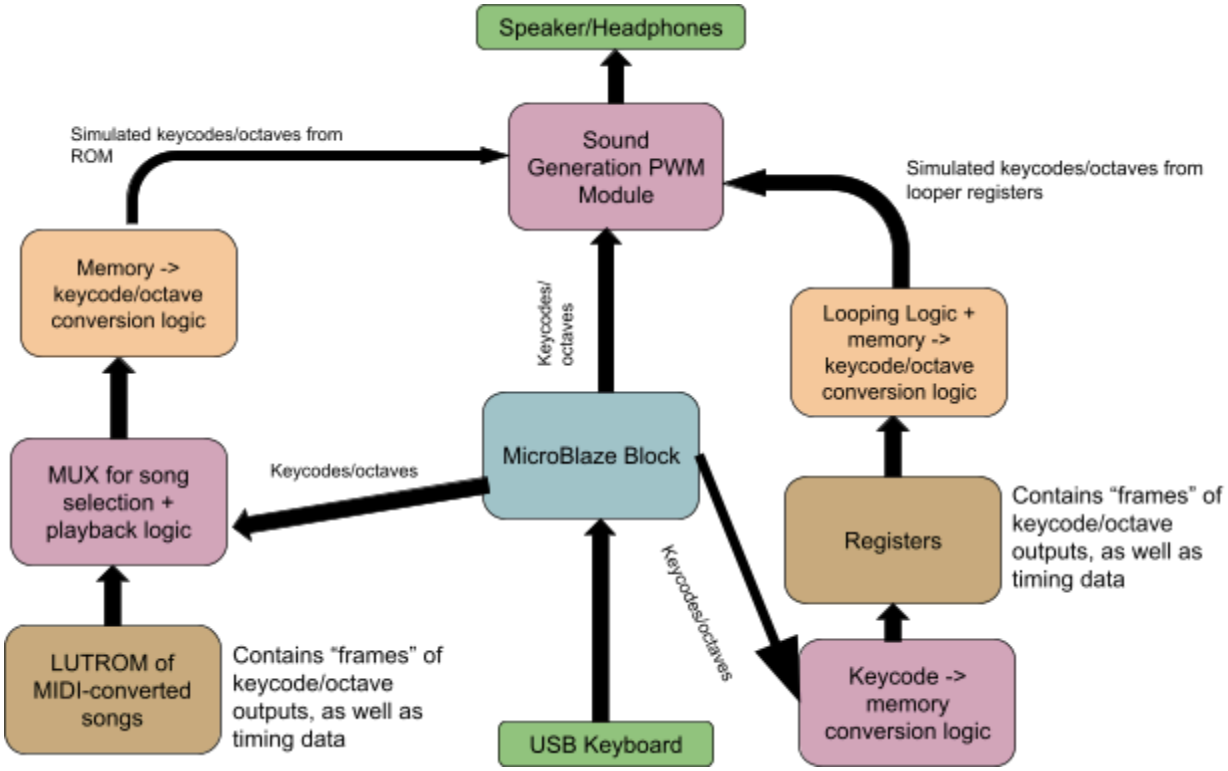
Pressing any of the NUMPAD keys 1-9, will each start playing a premade MIDI file that is loaded onto the board. Pressing the delete key (on NUMPAD) will stop the playback. Songs are loaded by taking a MIDI file, converting it into a format compatible with our memory and PWM generator using a Python script we made (detailed below), and then loading it into a .mem file in Vivado. The playback module then has logic to decode the .mem file and extract keycode, octave, and timing data, which is then fed into the PWM module. Essentially, the memory contains a series of timed key events which mimic a user actually playing on the synthesizer.

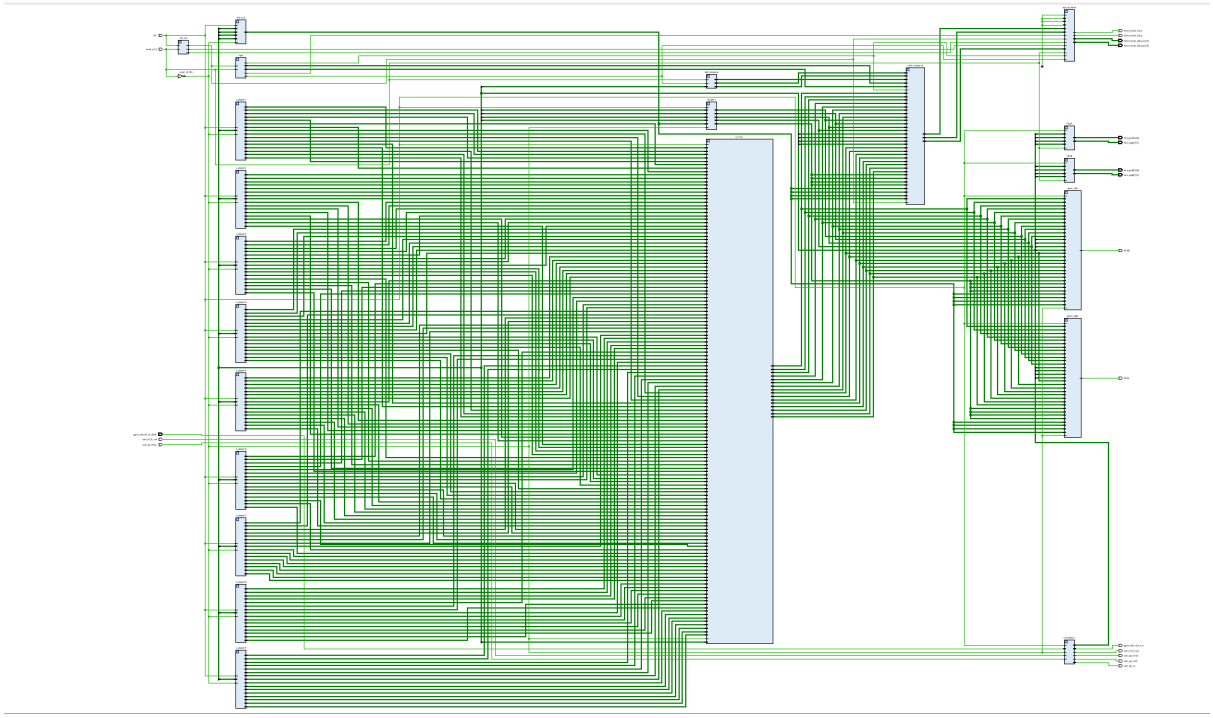
On-Screen Keyboard GUI:

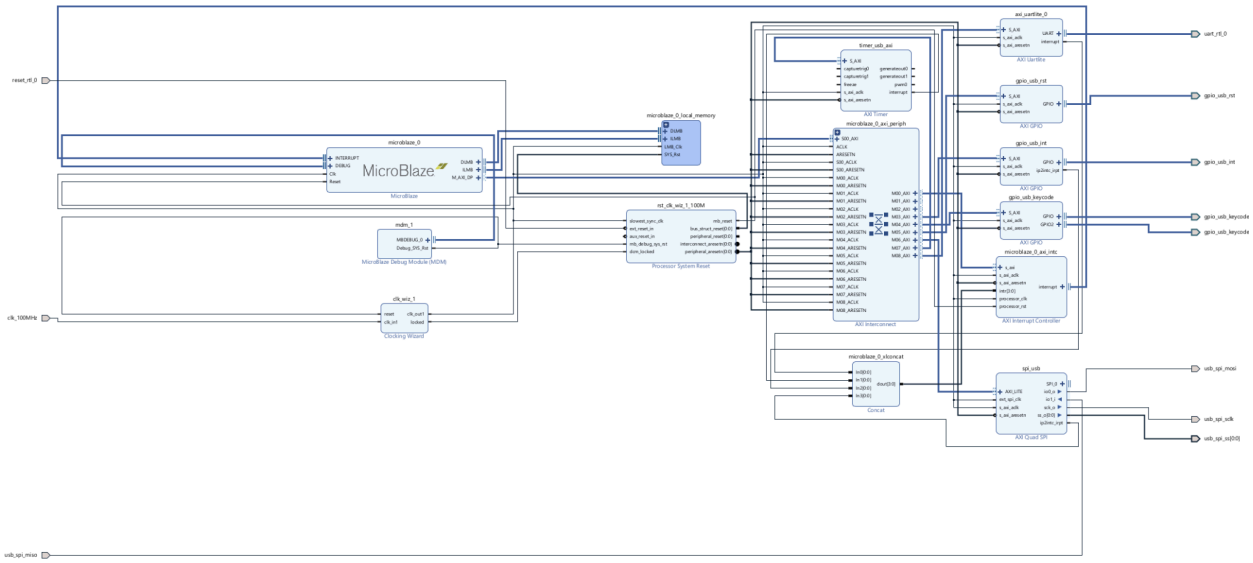
When plugged into an HDMI monitor, a virtual keyboard is displayed on screen. When notes are played, they are highlighted to the corresponding key on the keyboard. The GUI also

features a fun background depicting a staring man playing a piano, which is loaded from a BRAM generated by the image to COE script provided by the CAs. The GUI will highlight all keys that are being played by any mode: USB keyboard, looper, or MIDI playback.

Block Diagram







Explanation of Playing Notes + Polyphony

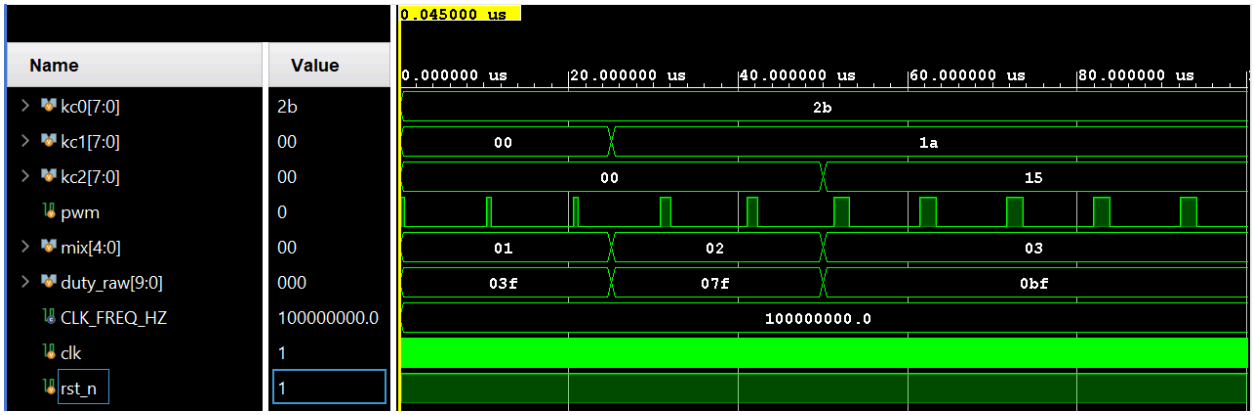
Every playable pitch can be defined as a combination of an 8-bit keycode and a 3-bit octave code. The keycodes are taken directly from lab 6.2, and simply correspond to whatever

keys are pressed on the USB keyboard. The octave can come from the octave selector (F1-F6 with F4 setting the leftmost key to middle C aka C4), the looper, or the ROM pre-recorded song file. Together they form a tuple of (keycode, octave), each of which represents one musical note. We decode the keycode in a 24 input lookup table that stores the half-period for the base octave, which is F4 through B5. Since moving up or down an octave is just multiplying frequency by 2 or ½ respectively, we can bitshift the frequencies to move octaves with minimal resources and without utilizing any digital signal processing techniques.

Each voice has its own square wave oscillator. With each 100 MHz rising edge, the counter increments. If the snapshot of notes changes in this time, the half-period is reinitialized in the same cycle. This logic is independent for each of the 16 copies, allowing us to play 16 independent notes at once, without interference. The way these notes are mixed is just a 5 bit sum of the 16 one-bit levels. Because each of the oscillations happen synchronously, the summed waveforms (multiple notes) are phase aligned, which enables the user and ROMS to play chords.

A counter that runs from 0 to 1023 gives us a 97.7kHz switch rate. The output which is pinned to the audio jack is high whenever counter < duty. This allows us to get a clean staircase shaped approximation of the combined square wave notes played from the audio jack. Since each oscillator is a 32 bit counter, a flip flop, and a comparator it only uses about 35 FF and a low amount of lookup tables for each. The cheapness of this hardware allows us to be able to play a pre-recorded song, a user-recorded loop, as well as real time note playing at the same time, without dropping notes.

Here is a simulation from Vivado with an explanation:



From times 0 to 25us, only kc0 is being played, which is 2B, corresponding to middle C. So only 1 voice is active and this can be seen in the skinniest 3 square waves at the start of the pwm waveform. From 25 to 50 us another note is being added, E. As you can see the duty doubles from 3f to 7f. This is reflected in the wider square waves in this period. Another note is added after this, G. As you can see the duty of this is BF. Duty is calculated by taking mix/16 * 1023. As you can see the square waves are even wider now. The pulse width widens step size as the new oscillator is toggled. This shows the summing of the square waves.

Explanation of "Pre-Recorded" Playback

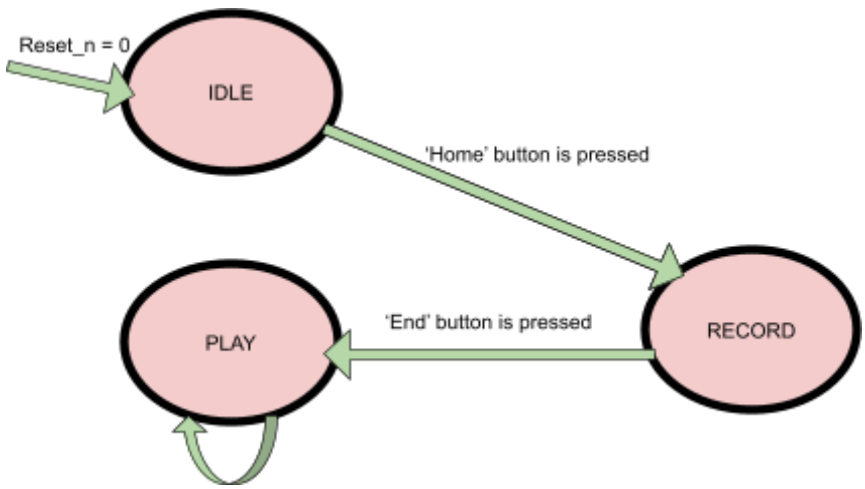
The MIDI format is similar to our looper format in that it consists of events with data such as pitch, whether the note is being toggled on or off, volume, etc. The format also has messages that specify tempo changes, metadata, synchronization, etc. None of the data is actual audio data; they are simply instructions for a MIDI compatible device, which then generates the sound. This means MIDI files are extremely space efficient. We used this principle to be able to store over 10 minutes of music at once on the board without an SD card. We were primarily interested in the note on and note off events in the MIDI files. To convert from MIDI to a format

compatible with our playback module, we made a Python script that parses the MIDI file, converts MIDI notes into our keycode/octave format, generates a new event when any keycode/octave is updated, and writes them all out into a .mem file that we can then load into our ROM. The format of the ROM is the first 30 bits are the delay between the current event and the previous event, in 100 MHz ticks. The next 88 bits are 8 pairs of (keycode, octave) tuples, since our playback module can play up to 8 concurrent notes. The playback module is a simple state machine that starts in an idle state and upon pressing the play button, transitions to the play state, which increments the tick counter until it matches the tick count in the event, then reads out that event's data into the keycode/octave outputs. It then moves on to the next event. This continues until the last event is reached or the end of the memory is reached, at which point the read pointer returns to 0.

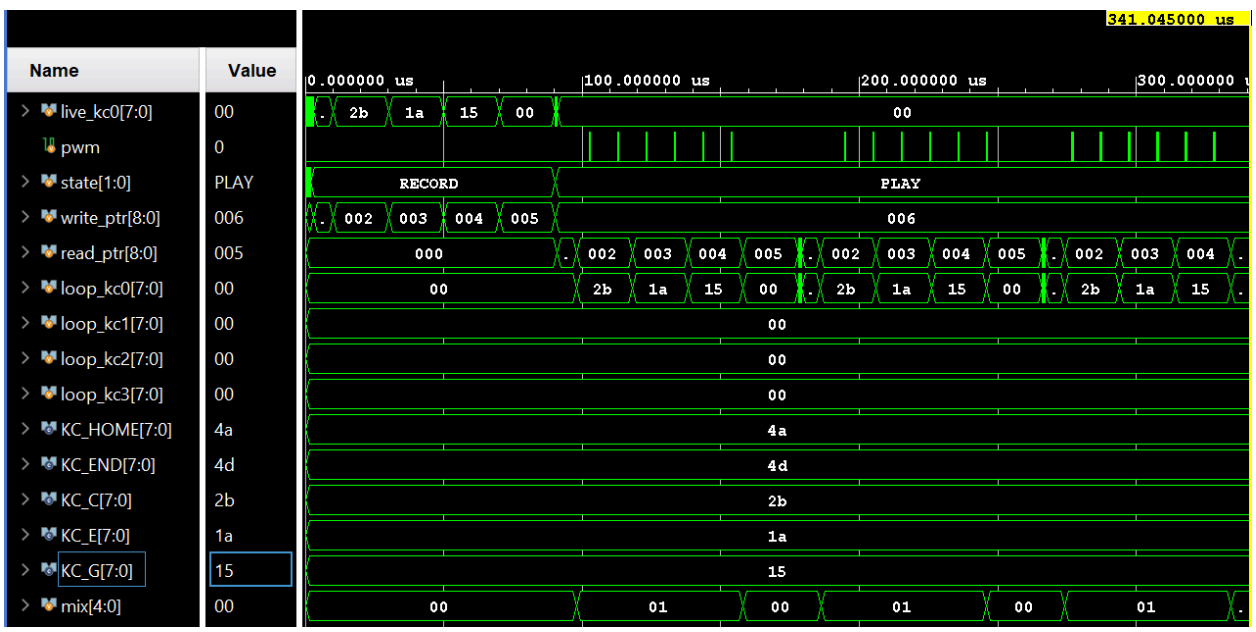
Explanation of Looping

When the 'Home' button is pressed on the keyboard, the looper FSM is put into RECORD mode. This captures the current USB input snapshot immediately so sustained notes are part of the first frame. While in record the looper records five signals: live_oct, live_kc0, live_kc1, live_kc2, and live_kc3. A 30 bit counter that runs at 100MHz measures the time before different events. At any point where any of the five signals differ from the previous snapshot, the looper stores the new delta tick time into BRAM, along with the keycodes and octaves at that moment, and clears the counter. For hardware limitation purposes, the 30 bit delta can store up to about 10 seconds between events without causing an overflow. The loop recording ends when the 'End' button on the keyboard is pressed. This resets the tick counter and loads the first stored

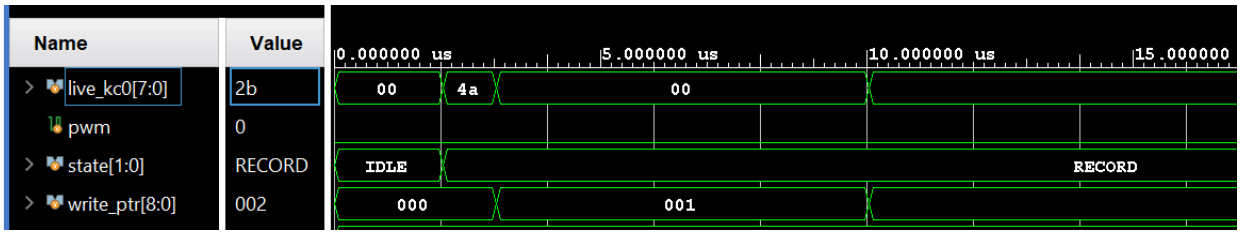
chord/note onto the loop output. In the PLAY state, the looper counts just like it did before and drives loop_oct and the loop keycodes with the stored snapshots. We used registers for the ease of combinational reads during playback. Our events are pretty space efficient, at 64*74 total bits, so we thought it was a fair tradeoff to simply use registers and not have to worry about BRAM timing.



Simulation with explanation:



Closer look at live keycodes:



As you can see, HOME is pressed, then a sequence of keys, then END. AFter that there are no live keys being played. However, the sequence recorded is being played as seen by the pwm waveform. You can also see this reflected in the state wave.

Explanation of Visual GUI

Starting with the background layer, we implemented this using the tutorial found in the course resources. We took an image we created and made it a COE, which we drew as the bottom-most layer. To construct the piano GUI, first the white keys from pixels 361 to 479 vertically are assigned to white. They have vertical separators every 13 pixels and a horizontal line at Y=360 to serve as the outlines. Black keys are about half the width of white keys (as they are on a real keyboard), and each octave pattern is repeated every 91 pixels. They are drawn on top of the white lines to create the overlapping look. This geometry is purely arithmetic meaning it uses zero BRAM. The corresponding colors are drawn in an always comb block, which, from bottom layer to top layer, draws the background, the white keys, the black keys and the separators, and the highlights of keys when played.

This same math is used to determine the pixels that each note belongs to. Every audio source used in this project (live playing, looping, and song player) can play up to 16 keys each

frame, each of them being a pair (keycode, octave). We made an 84-bit bitmap that records which notes are on, and this is updated in an always comb block that checks all 16 keycode octave pairs.

6. Design Resources and Statistics

LUT	29872
DSP	7
Memory (BRAM)	15.5
Flip-Flop	14506
Frequency	47.8 MHz
Static Power	.076 W
Dynamic Power	.421 W
Total Power	.497 W
WNS	-10.921 ns

Bugs/Errors Faced

We had two large bugs during the process of making the project. When we started implementing looping, we wanted to use BRAM and cross clock domains to a much slower frequency, around 400 Hz, for checking note presses and recording time between events. This created a lot of problems, as it was very difficult to get the clock edges of the 100 Mhz signal and the 400 Hz signal to match up, leading to a lot of dropped events and notes cutting in and out during playback. To fix this, we decided to massively simplify our approach and just use the system 100 Mhz clock for delta times and use registers for storing loop events. This came at a slight space

tradeoff, but made our logic a lot easier to debug since now everything was in the same clock domain.

Our second big challenge when debugging was figuring out why if we changed notes several times while holding down other notes, the sound would sometimes go silent. We determined that this was due to an edge case in our code where the wave oscillator would go silent if there was not at least one clock cycle of no keys being played when switching keys. This was rectified by creating a latching system where on every clock cycle, the previous set of keycodes was recorded and compared to the current keycodes, and if a change was detected, that square wave oscillator would instantly change to the new frequency and be set high. This fixed the cutting out issue because we now had well defined behavior for the previously mentioned edge case.

Conclusion:

With all of these components connected together, we were able to successfully implement the synthesizer and add a lot of memory based features for multiple playback modes. Some potential future feature additions would be more GUI flair, like the falling piano roll from Synthesia, as well as more waveforms, like sawtooth or triangle. Overall, it was a lot of fun figuring out how audio worked on the board, as it was not covered in any lab.

Appendix

Block diagram components (reused from lab 6.2):

MicroBlaze: A 32-bit RISC processor that can execute embedded C code. It interfaces via AXI to GPIOs and

MicroBlaze Debug Module (MDM): Provides debug access to the microblaze core without extra hardware.

Clocking Wizard: Generates clocks to produce various frequencies (25MHz, 125MHz, etc) based on input references.

Processor System Reset: Monitors reset inputs and clock locks to generate a synchronized reset signal for the microblaze and all AXI peripherals.

AXI Timer: 32-bit down/up counter with interrupt that is driven by the AXI bus. It

AXI Interconnect: Routs AXI translation between masters (MicroBlaze) and slaves(timers, GPIO, SPI, etc), handling address decoding.

Concat: Concatenates single-bit interrupt sources into a single bus so they can be imputed to the AXI Interrupt Controller.

AXI Uartlite: UART interface providing serial-port functions over AXI.

AXI GPIO: Exposes input and output pins to the processor via a memory mapped interface.

AXI Interrupt Controller: Collects and presents multiple interrupt lines by priority to the MicroBlaze through a single interrupt port.

AXI Quad SPI: Provides AXI controlled master interface for SPI. It talks to external SPI peripherals.

Background BRAM: BRAM module containing data used to draw our background sprite. Generated using the CA provided image to COE tool.

SV Modules:

vga_controller.sv:

Inputs: pixel_clk, reset

Outputs: hs, vs, active_nblank, sync, [9:0] drawX, [9:0] drawY

Description: Generates VGA timing signals for a 640x480 resolution display. It uses a pixel clock to increment horizontal (hc) and vertical (vc) counters. Based on these counters, it produces active-low horizontal sync (hs) and vertical sync (vs) pulses according to standard VGA timing for 800 total horizontal pixels and 525 total vertical lines. The active_nblank signal indicates when the current pixel coordinates (drawX, drawY) are within the visible 640x480 display area.

Purpose: To provide the synchronization signals (horizontal sync, vertical sync) and active pixel coordinates required to drive a VGA monitor or a display system that uses VGA timing, ensuring that an image is displayed correctly at a 640x480 resolution.

color_mapper.sv:

Inputs: [9:0] BallX, [9:0] BallY, [9:0] Ball_size, [9:0] DrawX, [9:0] DrawY, vga_clk, blank, [7:0] kc0, [7:0] kc1, [7:0] kc2, [7:0] kc3, [7:0] kc4, [7:0] kc5, [7:0] kc6, [7:0] kc7, [7:0] kc8, [7:0] kc9, [7:0] kc10, [7:0] kc11, [7:0] kc12, [7:0] kc13, [7:0] kc14, [7:0] kc15, [2:0] oct0, [2:0] oct1, [2:0] oct2, [2:0] oct3, [2:0] oct4, [2:0] oct5, [2:0] oct6, [2:0] oct7, [2:0] oct8, [2:0] oct9, [2:0] oct10, [2:0] oct11, [2:0] oct12, [2:0] oct13, [2:0] oct14, [2:0] oct15

Outputs: [3:0] Red, [3:0] Green, [3:0] Blue

Description: Determines the RGB color for each pixel on the screen. It draws a keyboard representation with active key highlighting based on 16 keycode/octave inputs, and integrates a background sprite. It decodes keycodes to musical notes and determines pixel colors based on a priority system (ball, active keys, keyboard graphics, background).

Purpose: To generate the visual output for a display, generate musical keyboard interface and background graphics, and to provide visual feedback for musical note input.

hex_driver.sv:

Inputs: clk, reset, [3:0] in[4]

Outputs: [7:0] hex_seg, [3:0] hex_grid

Description: Drives a 4-digit 7-segment hexadecimal display. It takes four 4-bit inputs, converts each nibble to its corresponding 7-segment pattern, and uses a counter to time-multiplex the display of each digit on the physical hex grid. Includes a sub-module nibble_to_hex for the conversion.

Purpose: Allows visual verification of four 4-bit data values (nibbles) on 7-segment displays, displaying keycodes for our project.

octave_selector.sv:

Inputs: clk, rst_n, [7:0] key0, [7:0] key1, [7:0] key2, [7:0] key3

Outputs: [2:0] octave

Description: Selects an octave value based on specific function key (F1-F6) presses detected from up to four keycode inputs. The selected octave is maintained until another recognized function key is pressed. Defaults to OCT4 on reset.

Purpose: To allow a user to change the current musical octave for live input using designated keyboard keys.

poly_note_pwm.sv:

Inputs: clk, rst_n, [7:0] key0, [7:0] key1, [7:0] key2, [7:0] key3, [7:0] key4, [7:0] key5, [7:0] key6, [7:0] key7, [7:0] key8, [7:0] key9, [7:0] key10, [7:0] key11, [7:0] key12, [7:0] key13, [7:0] key14, [7:0] key15, [2:0] octave0, [2:0] octave1, [2:0] octave2, [2:0] octave3, [2:0] octave4, [2:0] octave5, [2:0] octave6, [2:0] octave7, [2:0] octave8, [2:0] octave9, [2:0] octave10, [2:0] octave11, [2:0] octave12, [2:0] octave13, [2:0] octave14, [2:0] octave15

Outputs: pwm_out

Description: A 16-voice polyphonic square-wave synthesizer. It decodes 16 pairs of (keycode, octave) inputs into corresponding note frequencies (half-periods) using a look-up table. Each voice is an independent square-wave oscillator. The outputs of these oscillators are mixed, and the normalized sum is used to generate a Pulse Width Modulated (PWM) output signal.

Purpose: To generate an analog audio signal representing up to 16 simultaneous musical notes based on keyboard inputs, suitable for driving a speaker.

looper.sv:

Inputs: clk, rst_n, [7:0] live_kc0, [7:0] live_kc1, [7:0] live_kc2, [7:0] live_kc3, [2:0] live_oct

Outputs: [7:0] loop_kc0, [7:0] loop_kc1, [7:0] loop_kc2, [7:0] loop_kc3, [2:0] loop_oct

Description: A keyboard event looper that records and plays back a sequence of up to 4 simultaneous key presses and their associated octave. Recording is started by a 'HOME' key press and stopped by an 'END' key press. During recording, it stores the time delta between key state changes and the new key state (4 keycodes + octave) into an event RAM. During playback, it reads the events from RAM, waits for the stored delta time, and then outputs the recorded key state. Playback loops continuously.

Purpose: To enable recording and looping of musical phrases played on a USB keyboard, allowing for layered performances or accompaniment.

player_mux.sv:

Inputs: clk, rst_n, [7:0] sel_keycode, [7:0] in_kc[9][8], [2:0] in_oct[9][8]

Outputs: [7:0] out_kc[8], [2:0] out_oct[8]

Description: A multiplexer that selects one set of 8-voice keycode and octave data from 'N' possible input sets. The selection is controlled by sel_keycode. If sel_keycode matches one of the keycodes (parameter, an array of N keycodes), the corresponding in_kc and in_oct data is routed to out_kc and out_oct. If sel_keycode matches the delete key or on reset, the output is zeroed.

Purpose: To select one active "song" or "track" (represented by 8 voices of keycodes and octaves) from multiple available tracks based on a user's key press. It allows switching between different pre-programmed musical pieces.

player_rom.sv:

Inputs: [clog2(DEPTH)-1:0] addr (where DEPTH is a parameter)

Outputs: [117:0] data_out

Description: A Read-Only Memory module that stores musical event data. It is parameterized by DEPTH (number of entries in ROM) and ROM_FILE (the memory initialization file). On receiving an addr, it outputs the corresponding data word from the memory. The data word contains a time delta and 8 voices of keycode and octave information.

Purpose: To store pre-programmed musical sequences (songs) as a series of timed events, which can then be read out sequentially by a player module.

player.sv:

Parameters: DEPTH

Inputs: clk, rst_n, [7:0] live_kc0

Outputs: [7:0] playback_kc[8], [2:0] playback_oct[8]

Description: Plays back a pre-recorded 8-voice musical sequence from an associated player_rom instance. Playback is initiated when live_kc0 matches a specific NUMPAD_KEY (parameter). The module reads event words from the ROM, where each word contains a time delta and key/octave data for 8 voices. It waits for the specified delta time, then outputs the key/octave data. Playback loops when the end of the ROM sequence is reached. It uses helper functions idx8 and idx3 for bit-slicing the event word.
Purpose: To act as a music sequencer that reads and plays timed musical events (notes and octaves for 8 voices) stored in a ROM, triggered by a specific key press.

mb_usb_hdmi_top.sv:

Inputs: Clk, reset_rtl_0, gpio_usb_int_tri_i, usb_spi_miso, uart_rtl_0_rxd

Outputs: gpio_usb_rst_tri_o, usb_spi_mosi, usb_spi_sclk, usb_spi_ss, uart_rtl_0_txd, hdmi_tmds_clk_n, hdmi_tmds_clk_p, [2:0] hdmi_tmds_data_n, [2:0] hdmi_tmds_data_p, [7:0] hex_segA, [3:0] hex_gridA, [7:0] hex_segB, [3:0] hex_gridB, SPKR, SPKL

Description: Top-level module integrating USB keyboard input processing, HDMI video output generation, audio synthesis, a multi-channel ROM player, a looper, and hexadecimal display drivers. It manages clock generation for HDMI, VGA timing, and coordinates data flow between the USB interface, audio synthesizers (live, looped, ROM), color mapper for HDMI display, and hex displays for keycode visualization.

Purpose: Top level module for music synthesiser via USB keyboard, pre-recorded song playback, live looping functionality, visual feedback on an HDMI display, and audio output to speakers.